

Project Development – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 1 – Development Overview and Environment Setup

The Project Development phase converts the approved design and plan into a working EduGenie-AI application. Development focuses on implementing the FastAPI backend, connecting Google Gemini, creating learning modules, developing the web interface, and integrating all components.

Development Objectives

- Build a functional backend using FastAPI and Uvicorn.
- Connect the application to Google Gemini using the Google GenAI Python SDK.
- Implement question answering, explanation, quiz, summary, and learning recommendation features.
- Develop a simple HTML, CSS, and JavaScript frontend.
- Connect frontend requests with backend API endpoints.
- Handle configuration, errors, and API credentials securely.

Development Environment

Component	Technology / Purpose
Programming Language	Python
Backend Framework	FastAPI
Server	Uvicorn
AI Service	Google Gemini
AI SDK	Google GenAI Python SDK
Frontend	HTML, CSS, JavaScript
Development Tool	Visual Studio Code
Version Control	Git and GitHub
Configuration	.env environment variables

Initial Setup

Development begins by creating the project directory and Python virtual environment. Required packages are installed from requirements.txt. The Gemini API key and model name are stored in the .env file, while .gitignore prevents the secret configuration from being committed to the repository.

Page 2 – Backend and Gemini Integration

The backend is the central part of EduGenie-AI. FastAPI receives requests from the frontend, connects them to the correct learning module, and returns the generated result.

FastAPI Development

- Create the FastAPI application in main.py.
- Configure the templates directory for the web interface.
- Create routes for health checking and learning operations.
- Connect each route to the appropriate learning module.
- Return structured responses that can be processed by JavaScript.
- Provide useful error responses when a request cannot be completed.

Main Application Endpoints

Endpoint	Purpose
/	Loads the EduGenie-AI web interface
/health	Checks whether the application is running
/qa	Handles student question answering
/explain	Generates explanations for topics
/quiz	Generates quiz-oriented learning content
/summarize	Creates concise summaries
/learn/recommendations	Provides learning recommendations

Gemini Integration

The application uses the Google GenAI Python SDK to create a Gemini client. The API key is loaded from the environment configuration and the model is selected through GEMINI_MODEL. A task-specific prompt is created for each learning operation before sending the request to Gemini.

Question Answering Development

- Receive the student's question from the API request.
- Create an educational prompt with simple-language instructions.
- Send the prompt to the configured Gemini model.
- Read the generated response.
- Return the answer to the FastAPI route and frontend.

Page 3 – Learning Modules Development

EduGenie-AI uses separate modules so that each learning function can be developed and maintained independently. This modular approach makes debugging and future enhancement easier.

Module Development

Module	Development Activity	Expected Result
Q&A	Build question prompt and Gemini call	Educational answer
Explanation	Build concept explanation prompt	Step-by-step explanation
Quiz	Generate structured quiz content	Questions and answers
Summary	Process supplied learning text	Concise summary
Learning Path	Generate topic-based recommendations	Suggested learning sequence

Prompt Engineering

- Define EduGenie as a helpful learning assistant.
- Tell the model to use simple and clear language.
- Include the learner's actual input in the prompt.
- Specify the requested learning operation.
- Request useful structure such as headings, steps, examples, or quiz format.
- Avoid unnecessary information so responses remain focused.

Frontend Development

- Create the main HTML page for EduGenie-AI.
- Add input controls for questions, topics, and text.
- Create buttons or controls for the major learning features.
- Use CSS to make the interface readable and student-friendly.
- Use JavaScript fetch requests to communicate with FastAPI.
- Display returned AI content inside the result area.

Integration Development

After individual modules are implemented, they are connected to the FastAPI routes. The frontend then calls these routes and displays the returned results. Integration ensures that the complete flow—from student input to Gemini response—is working correctly.

Page 4 – Error Handling, Security and Debugging

During development, errors can occur because of incorrect input, configuration problems, network issues, unavailable AI services, or programming mistakes. EduGenie-AI therefore includes basic error handling and debugging practices.

Common Development Issues

Issue	Possible Cause	Development Response
Server does not start	Python/package/configuration issue	Check terminal error and dependencies
404 error	Incorrect route	Verify FastAPI endpoint
AI request fails	API key/model/service issue	Check .env and Gemini configuration
Empty result	Invalid input or response handling	Validate input and response processing
Frontend request fails	Wrong endpoint or JavaScript error	Check browser console and route
Unexpected AI response	Prompt not specific enough	Improve prompt instructions

Security Practices

- Keep GEMINI_API_KEY inside .env rather than source files.
- Do not expose the API key in frontend JavaScript.
- Add .env to .gitignore.
- Avoid printing credentials in terminal logs or error messages.
- Use environment variables for configurable model settings.
- Before publishing code, check that secret files are not staged in Git.

Debugging Process

- Read the exact error message from the terminal.
- Identify whether the problem is frontend, backend, configuration, or AI related.
- Test individual components separately.
- Use the /health endpoint to verify the server.
- Test Gemini independently when diagnosing AI connectivity.
- After fixing an issue, repeat the affected feature test.

Version Control

Git is used to track source-code changes during development. Meaningful commits help identify when a change was introduced and make it easier to maintain the project. Sensitive configuration such as .env should remain excluded from the repository.

Page 5 – Integration, Development Results and Future Improvements

The final development stage combines all completed components into one working learning assistant. The application is started with Uvicorn, accessed through a browser, and tested using the available learning features.

Development Completion Checklist

- FastAPI application starts successfully.
- The home page loads correctly.
- Health endpoint confirms application availability.
- Gemini API communication works with the configured model.
- Question answering feature produces responses.
- Explanation feature generates topic explanations.
- Quiz feature generates quiz content.
- Summary feature produces concise learning material.
- Learning recommendation feature returns useful suggestions.
- Frontend JavaScript communicates correctly with backend routes.
- API credentials remain protected from source control.

Development Results

The completed development work produces a modular AI learning assistant in which students can enter learning requests through a web interface and receive AI-generated educational content. FastAPI manages the application flow, while Google Gemini provides the natural-language generation capability.

Future Development Improvements

- Add user login and personalized student profiles.
- Store learning history and quiz performance in a database.
- Add progress tracking and dashboards.
- Support different difficulty levels.
- Add multilingual and voice-based learning features.
- Improve quiz evaluation and scoring.
- Add teacher-oriented controls and analytics.
- Introduce stronger validation and automated testing.

Conclusion

The Project Development phase transforms the EduGenie-AI design into a working application. The development process combines FastAPI, Google Gemini, modular Python components, and a web frontend. Secure configuration, modular coding, debugging, and integration practices provide a foundation for the following Testing, Documentation, and Demonstration phases.